

Are Deep Neural Networks Dramatically Overfitted?

This PDF conversion (v1) was generated on October 10, 2025*

Online version: <https://lilianweng.github.io/posts/2019-03-14-overfit/>

Date: **Estimated Reading Time:** min **Author:** Lilian Weng

⁰This file was prepared by Sakura (bili_sakura@zju.edu.cn). Please contact for any issues or copyright concerns. The original source of this file is <https://lilianweng.github.io/posts/2019-03-14-overfit/>.

Contents

1	Are Deep Neural Networks Dramatically Overfitted?	3
2	Classic Theorems on Compression and Model Selection#	4
2.1	Occam's Razor#	4
2.2	Minimum Description Length principle#	4
2.3	Kolmogorov Complexity#	5
2.4	Solomonoff's Inference Theory#	6
3	Expressive Power of DL Models#	6
3.1	Universal Approximation Theorem#	6
3.2	Proof: Finite Sample Expressivity of Two-layer NN#	6
7	Citation	7
3.3	Deep NN can Learn Random Noise#	8
4	Are Deep Learning Models Dramatically Overfitted?#	8
4.1	Modern Risk Curve for Deep Learning#	8
4.2	Regularization is not the Key to Generalization#	8
4.3	Intrinsic Dimension#	8
4.4	Heterogeneous Layer Robustness#	8
4.5	The Lottery Ticket Hypothesis#	8
5	Experiments#	8
6	References#	8
8	References	10

[Lil'Log](#)

- [|](#)
- [Posts](#)
- [Archive](#)
- [Search](#)
- [Tags](#)
- [FAQ](#)

1 Are Deep Neural Networks Dramatically Overfitted?

Date: March 14, 2019 | Estimated Reading Time: 21 min | Author: Lilian Weng

[Table of Contents](#)

- [Classic Theorems on Compression and Model Selection](#)
 - [Occam's Razor](#)
 - [Minimum Description Length principle](#)
 - [Kolmogorov Complexity](#)
 - [Solomonoff's Inference Theory](#)
- [Expressive Power of DL Models](#)
 - [Universal Approximation Theorem](#)
 - [Proof: Finite Sample Expressivity of Two-layer NN](#)
 - [Deep NN can Learn Random Noise](#)
- [Are Deep Learning Models Dramatically Overfitted?](#)
 - [Modern Risk Curve for Deep Learning](#)
 - [Regularization is not the Key to Generalization](#)
 - [Intrinsic Dimension](#)
 - [Heterogeneous Layer Robustness](#)
 - [The Lottery Ticket Hypothesis](#)
- [Experiments](#)
- [References](#)

[Updated on 2019-05-27: add the [section](#) on Lottery Ticket Hypothesis.]

If you are like me, entering into the field of deep learning with experience in traditional machine learning, you may often ponder over this question: Since a typical deep neural network has so many parameters and training error can easily be perfect, it should surely suffer from substantial overfitting. How could it be ever generalized to out-of-sample data points?

The effort in understanding why deep neural networks can generalize somehow reminds me of this interesting paper on System Biology — “[Can a biologist fix a radio?](#)” (Lazebnik, 2002). If a biologist intends to fix a radio machine like how she works on a biological system, life could be hard. Because the full mechanism of the radio system is not revealed, poking small local functionalities might give some hints but it can hardly present all the interactions within the system, let alone the entire working flow. No matter whether you think it is relevant to DL, it is a very fun read.

I would like to discuss a couple of papers on generalizability and complexity measurement of deep learning models in the post. Hopefully, it could shed light on your thinking path towards the understanding of why DNN can generalize.

2 Classic Theorems on Compression and Model Selection#

Let’s say we have a classification problem and a dataset, we can develop many models to solve it, from fitting a simple linear regression to memorizing the full dataset in disk space. Which one is better? If we only care about the accuracy over training data (especially given that testing data is likely unknown), the memorization approach seems to be the best — well, it doesn’t sound right.

There are many classic theorems to guide us when deciding what types of properties a good model should possess in such scenarios.

2.1 Occam’s Razor#

[Occam’s Razor](#) is an informal principle for problem-solving, proposed by [William of Ockham](#) in the 14th century:

“Simpler solutions are more likely to be correct than complex ones.”

The statement is extremely powerful when we are facing multiple candidates of underlying theories to explain the world and have to pick one. Too many unnecessary assumptions might seem to be plausible for one problem, but harder to be generalized to other complications or to eventually lead to the basic principles of the universe.

Think of this, it took people hundreds of years to figure out that the sky is blue in the daytime but reddish at sunset are because of the same reason ([Rayleigh scattering](#)), although two phenomena look very different. People must have proposed many other explanations for them separately but the unified and simple version won eventually.

2.2 Minimum Description Length principle#

The principle of Occam’s Razor can be similarly applied to machine learning models. A formalized version of such concept is called the *Minimum Description Length (MDL)*

principle, used for comparing competing models / explanations given data observed.

“Comprehension is compression.”

The fundamental idea in MDL is to *view learning as data compression*. By compressing the data, we need to discover regularity or patterns in the data with the high potentiality to generalize to unseen samples. [Information bottleneck](#) theory believes that a deep neural network is trained first to represent the data by minimizing the generalization error and then learn to compress this representation by trimming noise.

Meanwhile, MDL considers the model description as part of the compression delivery, so the model cannot be arbitrarily large.

A *two-part version* of MDL principle states that: Let $\mathcal{H}^{\{(1)\}}, \mathcal{H}^{\{(2)\}}, \dots$ be a list of models that can explain the dataset $\mathcal{D}$. The best hypothesis among them should be the one that minimizes the sum:

$$\mathcal{H}^{\text{best}} = \arg\min_{\mathcal{H}} [L(\mathcal{H}) + L(\mathcal{D}|\mathcal{H})]$$

- $L(\mathcal{H})$ is the length of the description of model $\mathcal{H}$ in bits.
- $L(\mathcal{D}|\mathcal{H})$ is the length of the description of the data $\mathcal{D}$ in bits when encoded with $\mathcal{H}$.

In simple words, the *best* model is the *smallest* model containing the encoded data and the model itself. Following this criterion, the memorization approach I proposed at the beginning of the section sounds horrible no matter how good accuracy it can achieve on the training data.

People might argue Occam’s Razor is wrong, as given the real world can be arbitrarily complicated, why do we have to find simple models? One interesting view by MDL is to consider models as “**languages**” instead of fundamental generative theorems. We would like to find good compression strategies to describe regularity in a small set of samples, and they **do not have to be the “real” generative model** for explaining the phenomenon. Models can be wrong but still useful (i.e., think of any Bayesian prior).

2.3 Kolmogorov Complexity#

Kolmogorov Complexity relies on the concept of modern computers to define the algorithmic (descriptive) complexity of an object: It is *the length of the shortest binary computer program that describes the object*. Following MDL, a computer is essentially the most general form of data decompressor.

The formal definition of Kolmogorov Complexity states that: Given a universal computer $\mathcal{U}$ and a program p , let’s denote $\mathcal{U}(p)$ as the output of the computer processing the program and $L(p)$ as the descriptive length of the program. Then Kolmogorov Complexity $K_{\mathcal{U}}$ of a string s with respect to a universal computer $\mathcal{U}$ is:

$$K_{\mathcal{U}}(s) = \min_{\{p: \mathcal{U}(p)=s\}} L(p)$$

Note that a universal computer is one that can mimic the actions of any other computers. All modern computers are universal as they can all be reduced to Turing machines. The definition is universal no matter which computers we are using, because another universal computer can always be programmed to clone the behavior of $\mathcal{U}$, while encoding this clone program is just a constant.

There are a lot of connections between Kolmogorov Complexity and Shannon Information Theory, as both are tied to universal coding. It is an amazing fact that the expected Kolmogorov Complexity of a random variable is approximately equal to its Shannon entropy (see Sec 2.3 of [the report](#)). More on this topic is out of the scope here, but there are many interesting readings online. Help yourself :)

2.4 Solomonoff's Inference Theory#

Another mathematical formalization of Occam's Razor is Solomonoff's theory of universal inductive inference ([Solomonoff, 1964](#)). The principle is to favor models that correspond to the "shortest program" to produce the training data, based on its Kolmogorov complexity

3 Expressive Power of DL Models#

Deep neural networks have an extremely large number of parameters compared to the traditional statistical models. If we use MDL to measure the complexity of a deep neural network and consider the number of parameters as the model description length, it would look awful. The model description $L(\mathcal{H})$ can easily grow out of control.

However, having numerous parameters is *necessary* for a neural network to obtain high expressivity power. Because of its great capability to capture any flexible data representation, deep neural networks have achieved great success in many applications.

3.1 Universal Approximation Theorem#

The *Universal Approximation Theorem* states that a feedforward network with: 1) a linear output layer, 2) at least one hidden layer containing a finite number of neurons and 3) some activation function can approximate **any** continuous functions on a compact subset of $\mathbb{R}^n$ to arbitrary accuracy. The theorem was first proved for sigmoid activation function ([Cybenko, 1989](#)). Later it was shown that the universal approximation property is not specific to the choice of activation ([Hornik, 1991](#)) but the multilayer feedforward architecture.

Although a feedforward network with a single layer is sufficient to represent any function, the width has to be exponentially large. The universal approximation theorem does not guarantee whether the model can be learned or generalized properly. Often, adding more layers helps to reduce the number of hidden neurons needed in a shallow network.

To take advantage of the universal approximation theorem, we can always find a neural network to represent the target function with error under any desired threshold, but we need to pay the price — the network might grow super large.

3.2 Proof: Finite Sample Expressivity of Two-layer NN#

The Universal Approximation Theorem we have discussed so far does not consider a finite sample set. [Zhang, et al. \(2017\)](#) provided a neat proof on the finite-sample expressivity of two-layer neural networks.

A neural network $\mathcal{C}$ can represent any function given a sample size n in d dimensions if: For every finite sample set $S \subseteq \mathbb{R}^d$ with $|S| = n$

and every function defined on this sample set: $f: S \mapsto \mathbb{R}$, we can find a set of weight configuration for C so that $C(\mathbf{x}) = f(\mathbf{x})$, $\forall \mathbf{x} \in S$.

The paper proposed a theorem:

There exists a two-layer neural network with ReLU activations and $2n + d$ weights that can represent any function on a sample of size n in d dimensions.

Proof. First we would like to construct a two-layer neural network $C: \mathbb{R}^d \mapsto \mathbb{R}$. The input is a d -dimensional vector, $\mathbf{x} \in \mathbb{R}^d$. The hidden layer has h hidden units, associated with a weight matrix $\mathbf{W} \in \mathbb{R}^{d \times h}$, a bias vector $\mathbf{b} \in \mathbb{R}^h$ and ReLU activation function. The second layer outputs a scalar value with weight vector $\mathbf{v} \in \mathbb{R}^h$ and zero biases.

The output of network C for a input vector $\mathbf{x}$ can be represented as follows:

$$C(\mathbf{x}) = \mathbf{v} \cdot \max\{\mathbf{x} \cdot \mathbf{W} - \mathbf{b}, 0\}^{\text{top}} = \sum_{i=1}^h v_i \max\{\mathbf{x} \cdot \mathbf{W}_{(:,i)} - b_i, 0\}$$

where $\mathbf{W}_{(:,i)}$ is the i -th column in the $d \times h$ matrix.

Given a sample set $S = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ and target values $\mathbf{y} = \{y_1, \dots, y_n\}$, we would like to find proper weights $\mathbf{W} \in \mathbb{R}^{d \times h}$, $\mathbf{b}$, $\mathbf{v} \in \mathbb{R}^h$ so that $C(\mathbf{x}_i) = y_i$, $\forall i=1, \dots, n$.

Let's combine all sample points into one batch as one input matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$. If set $h=n$, $\mathbf{X} \cdot \mathbf{W} - \mathbf{b}$ would be a square matrix of size $n \times n$.

$$\mathbf{M} = \text{ReLU}(\mathbf{X} \cdot \mathbf{W} - \mathbf{b}), \quad 0 \leq \mathbf{M} = \begin{bmatrix} \mathbf{x}_1 \cdot \mathbf{W} - \mathbf{b} & \dots & \\ \mathbf{x}_n \cdot \mathbf{W} - \mathbf{b} & & \end{bmatrix} = [\mathbf{x}_i \cdot \mathbf{W}_{(:,j)} - b_j]_{i \times j}$$

We can simplify $\mathbf{W}$ to have the same column vectors across all the columns:

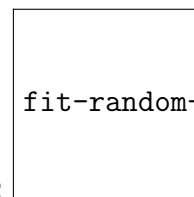
$$\mathbf{W}_{(:,j)} = \mathbf{w} \in \mathbb{R}^d, \quad \forall j = 1, \dots, n$$

7 Citation

Please cite this work as:

Weng, Lilian. "Title". Lil 'Log (May 2025). <https://lilianweng.github.io/posts/2019-03-14-overfit/>

Or use the BibTeX citation:



Let $\mathbf{M}$ be a $n \times n$ matrix. We can learn a non-linear function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ by minimizing the squared error $\| \mathbf{y} - \mathbf{f}(\mathbf{x}) \|^2$ over a set of training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$. This is a non-convex optimization problem, but it can be solved by gradient descent. The resulting function f is called a **Deep Neural Network** (DNN). DNNs are a class of artificial neural networks that consist of multiple layers of nodes. The input layer is connected to the hidden layers, which are connected to the output layer. The nodes in each layer are connected to the nodes in the next layer. The weights between the nodes are learned from the training data. DNNs are used for a wide range of tasks, including image recognition, speech recognition, and natural language processing.


```
@article{weng2025,  
title = {},  
author = {Weng, Lilian},  
journal = {lilianweng.github.io},  
year = {2025}, month = {May},  
url = {}  
}
```

8 References

Tags: [tag1](#) [tag2](#)

[⟨ Title](#) [Title ⟩](#)

Share this article: [Twitter](#) | [LinkedIn](#) | [Reddit](#) | [Facebook](#) | [WhatsApp](#) | [Telegram](#)